Detecting Fraudulent Online Job Postings Using Combined Text and Metadata Machine Learning

Yogeshwor Prasad Kharel
Wright State University
UID: U01127440

Akhalesh Sah
Wright State University
UID: U01123165

***Abstract*—Fraudulent postings on online employment sites have been one of the most common types of scams, in which scam artists use fake job ads to obtain individual's private data and to profit from elicit money. Fraud detection can be considered as a binary classification problem on the "Real or Fake Job Posting Prediction" dataset. This dataset includes 17,880 posting. However, only approximately 4.8% of these postings were deemed by the authors to be false. In this research we compare three families of models: (i) Text-only classifiers using TF-IDF features, (ii) Metadata-only tree-based classifiers using engineered structured features, and (iii) A fused model combining the two. All configurations utilize layered five fold-cross validation. Accuracy, Precision, Recall, F1, and ROC-AUC will be reported. The combined linear SVM produced the best overall results with an F1 value of 0.724 and an ROC-AUC of 0.986. Additionally, a random forest classifier trained solely on the metadata features resulted in an F1 value of 0.725. An ablation study was performed to determine whether text and metadata provide unique fraud signals. Finally, we conclude with an error analysis and a discussion regarding our limitation.**

***Index Terms*—fraud detection, text classification, TF-IDF, class imbalance, ensemble methods, ablation study**

GitHub: https://github.com/AYGOYUS/fake-job-posting-detection_ML_Final-Proj

I. Introduction

Online job boards serve as an important infrastructure to support the current labor market; however, online job boards have increasingly been targeted by fraudsters. Fraudulent postings are often utilized to obtain sensitive candidate information via phishing attempts or to collect application fees from candidates. These types of posting may be in advancement of money laundering schemes that involve the recruitment of unwitting candidates. Automated methods are necessary due to the sheer volume of postings made available through job boards. However, the problem is two-fold. First, fraudulent postings account for just a small percentage of total job postings. Second, many times, fraudulent postings appear to be nearly identical in appearance to legitimate job postings.

The dataset we used in our research is the publicly accessible Real or Fake Job Posting Prediction dataset [1]. This dataset includes 17,880 job postings that have been provided using 18 features, 866 of these postings were labeled as fraudulent (or 4.8% of total postings). The majority of the features represent free form text including title, company profile, description, required skills/benefits, etc., while others are structured data representing metadata like location, department, salary range, type of employment/contract, industry, required education/experience, and three other binary flags.

Fraudulent posting detection will be treated as a supervised binary classification problem. Our objectives include:

(i) creating independent text only and metadata only baseline models to make it easier to see what contributions each modality has;

(ii) building a fused model that utilizes both modalities and evaluate the amount of increase through an ablation analysis;

(iii) reporting evaluation results using metrics suitable for problems where there are large differences in class sizes.

Contributions.

- We create a completely explicit three-way ablation to separate the contributions from the different signal types text, metadata, and fused. This shows that these signals are complementary rather than redundant.
- We test nine classifier models (Linear Regression, Naive Bayes, Linear-SVM, Decision Tree, Random Forest, Gradient Boosting Model, and three combined models) with a single stratified five fold cross validation, while comparing both metrics, ROC-AUC as well as F1 which is often omitted in prior literature due to the large amount of imbalanced classes.
- To further build upon our comparison we have added explainability layers, including Chi-Square based most discriminative words for each model's output, metadata-based importance weights for each feature, and finally we report results of an error analysis where we identify the residual class of fake items missed by all of the tested models.

II. Related Work

A. Supervised Learning on Job-Posting Fraud

The classical supervised classifiers that were used to study the Real or Fake Job Posting Prediction dataset are numerous. Bansal's first release utilized an engineered Random Forest classifier for his baseline model and established the use of "presence-of X" indicator variables (does the post contain a company profile, logo, salary range) as primary signals. [1] Habiba, Islam, and Tasnim performed a comparison of Naive Bayes, K-Nearest Neighbors (KNN), Decision Tree, Random Forest and Multi-Layer Perceptron for this data set based upon TF-IDF feature extraction. The authors found that all group

of methods outperformed individual classifiers. However, they also found that class imbalances needed to be addressed in some manner. Our findings confirm those of the authors: when using only metadata as a feature vector, we were able to achieve F1 scores of approximately 0.72 for both Random Forest and Gradient Boosting, while the best single classifier Decision Tree was achieving only about 0.50. [2] Amaar et al. and Vajjala et al. pushed performance even higher using both a resampling strategy and text feature extraction. Amaar et al. employed ADASYN (Adaptive Synthetic Sampling) to oversample the underrepresented class while employing TF-IDF (Term Frequency-Inverse Document Frequency) as its feature representations. They found that although an Extra-Trees Classifier can produce extremely accurate results when creating a balanced dataset from the original unbalanced one. However, they determined that the resampling methodology is far more important than the type of classifier chosen after an effective feature representation is created. In our experiments we employ class_weight = "balanced" within scikit-learn. This provides us with an equivalent result to an explicit resampling method, but does so in such a way that it will not inflate the size of our dataset. [3] [4]

We differ from previous studies in two different ways. First, previous studies have typically used one of the families of features (either text or metadata) to determine the best classifier for those features. Whereas we determined how well nine classifiers would do using a unified layered five-fold cross validation protocol and ran an explicit ablation study to isolate the contribution of each modality. Second, we reported both F1 and ROC-AUC on our results; whereas many previous studies omitted the ROC-AUC metric when reporting their results since they were working with very unbalanced datasets.

B. Deep Sequence Models on This Dataset

In addition to traditional classical supervised machine learning techniques, researchers have been using new deep sequence models to apply these to the same data set. For example, Pillai et al. [5] built a Bidirectional Long Short-Term Memory (LSTM) model to process the description column of their data set. They also included a second input head which is used as a numerical input for processing the metadata in their model. The authors were able to achieve some very good results. These types of studies and other related deep learning submissions on Kaggle demonstrate how creating an appropriate contextual representation of the description column through applying deep learning methods could be effective in capturing the phrasing patterns of descriptions that would otherwise be missed by standard Term Frequency-Inverse Document Frequency (TF-IDF).

We deliberately have not used a deep model in this work. Our primary objective is to provide a transparent and reproducible comparison between standard classifiers as well as an abalation analysis of signals from both metadata and text. We believe that a small dataset (17,880 postings with ony 866 fraudulent) makes it difficult to interpret how pretrained transformers handle such bias. Nonetheless, the gains reported by deep sequence models indicate clear directions for future development which we will discuss in Section VI.

C. Positioning of Our Work

Rather than developing a new model our contributions are based upon a very detailed comparison. This includes evaluating nine different types of classifiers under the same stratified five fold evaluation protocol. Additionally we conducted an explicit three way ablation study, using the same base classifier for each condition (i.e., using the same model), where we used two separate sets of features: (i) text-only features, and (ii) metadata-only features. In doing so, differences in performance could be attributed to the characteristics of the data features rather than the choice of learning model. Additionally we reported both ROC-AUC scores as well as F1 scores, while some previous studies have omitted reporting F1 scores due to the fact that the class balance is highly skewed (approximately 21:1). Furthermore, we provided explanations for why certain numbers were produced through multiple means beyond just a single number. We utilized Chi-Squared values for determining the most discriminating words, determined the importance of individual metadata feature inputs to the model, and conducted an error analysis of those fake samples which all models incorrectly classified.

III. Methodology

A. Problem Formulation

Let $x_i = (x_i^{\text{text}}, x_i^{\text{meta}})$ be defined as job postings with both a text component and a metadata component, and let $y_i \in \{0, 1\}$ be defined as either "0" or "1" depending on if the post is legitimate or fake. We have access to a dataset of labeled posts, $\{(x_i, y_i)\}_{i=1}^{N}$, Thus we can train our classifier, $f_\theta : x \to \hat{y}$ based on minimizing a weighted cross entropy loss function with inverse class-weighted proportionally to each class distribution. This is useful because our training data has an extremely high ratio of legitimate to fake postings approximately 21:1.

B. Text Pipeline

Five separate text columns (title, company profile, description, requirements, benefits) were combined row by row into a single column. The cleaning process for this data was as follows: Lowercased; removed URLS; removed all non-alphabetical characters; removed stopworded english terms from the default set provided in scikit-learn; removed any word which had fewer than 3 alphabetical characters. This cleaned data was then converted into a numerical format with term-frequency/inverse-document-frequency (TF-IDF) using unigram and bigram tokenization. Additionally, there was an upper limit on the number of possible features at 5,000 and an upper/lower bound on the minimum/maximum document frequency (df) at 2 and 0.95 respectively. Following TF-IDF we applied χ^2-based feature selection using SelectKBest with $k = 2{,}000$), where the goal is to eliminate any features that have little or no bearing on classification performance.

C. Metadata Pipeline

The metadata columns in the dataset are automatically detected using the data-type of each column and the average number of characters per string to avoid specifying specific column-names within the pipeline. All three binary flags "telecommuting, has_company_logo, has_questions" remain unchanged, with missing values replaced with a value of 0. Categorical columns will have the string 'Unknown' used where there is a missing value and will then be encoded. In addition, we include an additional set of "has_X" indicator variables for all text and categorical columns which indicate if a given variable contains data. It was discovered that fraud can often be predicted based on the occurrence or lack thereof of missing data (Section V).

D. Combined Pipeline

To build the combined model we first concatenate the sparse TF-IDF matrix with the dense metadata matrix to get one horizontal block of data using hstack from scipy.sparse. Then we train one classifier that can learn both the interaction between text features and structural features in one step.

E. Models and Hyperparameters

The results include three models trained only on the text and three models trained only on the metadata. The default hyperparameters were used for all but when stated otherwise. All tree-based learners were based upon Random Forests and Gradient Boosting. Also included was a baseline that uses a linear classifier as a combination of TF-IDF and classifier on text. [7] [8] [9] [6].

Text-only models. Logistic Regression: $C = 1.0$, `liblinear` solver, `class_weight = balanced`. Multinomial Naive Bayes: smoothing $\alpha = 1.0$. Linear SVM: $C = 1.0$, `class_weight = balanced`.

Metadata-only models. Decision Tree: `max_depth = 10`, `class_weight = balanced`. Random Forest: 100 trees, `max_depth = 15`, `class_weight = balanced`. Gradient Boosting: 100 estimators, `max_depth = 5`, `learning_rate = 0.1`.

Combined models. Logistic Regression and Linear SVM with the same settings as their text-only versions and a Random Forest with `max_depth = 20`.

A coarse `GridSearchCV` was run over $C \in \{0.5, 1.0, 2.0\}$ for logistic regression and over `n_estimators` $\in \{100, 200\}$ and `max_depth` $\in \{10, 15, 20\}$ for random forest. F1 moved by less than .0.01 as well as results were very close to each other on both metrics so we selected the default settings listed above.

IV. Experimental Setup

A. Dataset and Splits

We utilize all 17,880 postings of the dataset to support our modeling choices. For both model selection and the layered five-fold cross validation used to maintain the fraud ratio at 4.8% we have also taken care to keep this same ratio consistent within each fold. In addition to the stratification described above, we also create a stratified test set with a train/test split of 80/20 for the confusion matrices, ROC curves and error analysis described in section V. The pipeline includes the ability to split data into a 70/15/15 training/testing/validation split by using the splits.py utility, and it will persist these indices for later use.

B. Exploratory Data Analysis

Since there are 17,014 legitimate posts and 866 fraudulent posts (Fig. 1), this causes severe skewness in the classes. Therefore it's inappropriate to rely on accuracy as a main metric because you can get about 95% accurate by simply predicting the majority class with no recall for fraud.

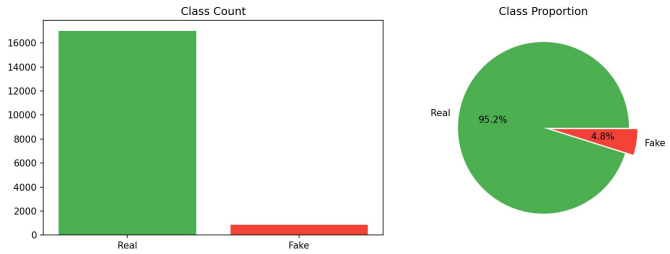

Fig. 1. Class distribution - Only 4.8 percent of postings are fraudulent.

(Fig. 2) and `department` are very infrequently populated (missing in 84% and 65%, respectively) and are therefore "optional" data elements. Fortunately, however, `description` will almost certainly be populated for every record because this field provides by far the largest text-based signals.

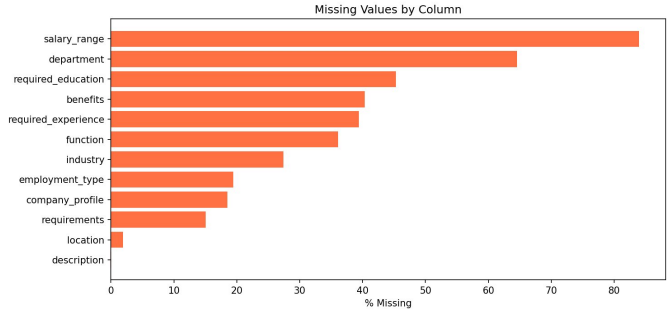

Fig. 2. Rate of Missing Values by Column. More than 40% of the columns for several optional metadata fields have missing values.

The length of real and fake postings differs in terms of how many characters appear in Fig. 3). The length of the `company_profile` section for real postings were approximately three times the length of those from fake postings. Similarly, `requirements` sections of real postings were also significantly longer than those for fake postings. Overall, it seems that fake postings have very short and generic text.

The most significant predictor for identifying the fraud in postings based on metadata appears to be the inclusion of a company's profile. This can be seen in the figure below where the correlation of each engineered feature with the target variable was calculated (Fig. 4. Both `has_company_profile`

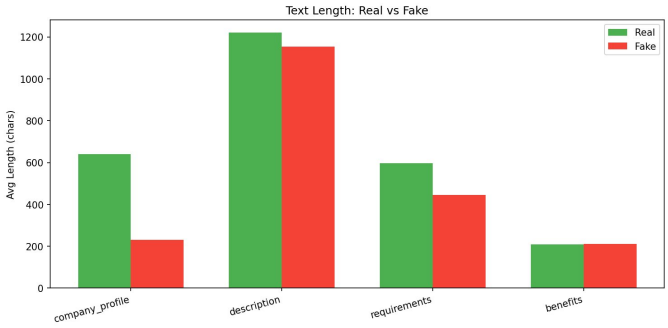

Fig. 3. Average character length of each text field, real versus fake.

and `has_company_logo` had a correlation with fraud that ranged from -0.25 to -0.29. Therefore, legitimate postings are much more likely to include these features. Consistent with the previous analysis, `telecommuting` has a weakly positive correlation with fraud.

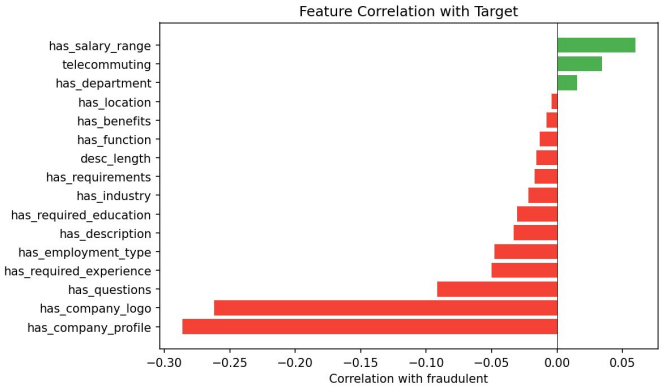

Fig. 4. Pearson correlation between "engineered" feature and fraud label. Bars to the left (negative), in red, represent how much each engineered feature is present when posting as an actual individual.

To further investigate which words were the most discriminative of classes we used the χ^2 test versus the fraud label on the TF-IDF engineered features. As shown in Fig. 5 the fraud group includes the terms *data entry*, *typing*, *earn*, *work home*, *cash*, and *oil gas*. This term has been a well-known scam within this dataset vertical..

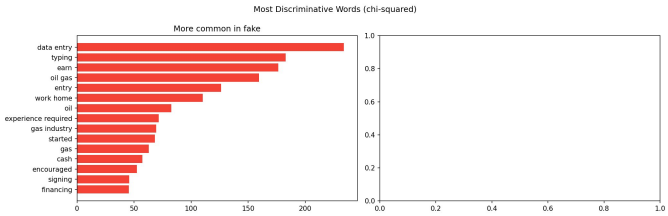

Fig. 5. Top tokens by χ^2 score for fraudulent postings.

C. Evaluation Metrics

The data we have is very imbalanced, just looking at how many postings were correctly labeled as fraudulent (accuracy) does not tell us much.

1) **Precision**: The proportion of posts which were identified by the model as fraudulent and actually are.
2) **Recall**: The proportion of truly fraudulent posts which the model identifies.
3) **F1 Score**: A single number that represents both precision and recall; it is a combination of the two values using their harmonic means. This is our key metric for comparing models.
4) **ROC-AUC**. An area under curve statistic that measures how well a classification model can distinguish between cases with and without a certain characteristic (in this case fraud). It provides an independent way to evaluate a classifier's performance regardless of the decision threshold used.
5) **Confusion Matrix**. To provide additional information about misclassifications made by the model.

D. Implementation

The data processing pipeline was created using Python (version 3.14) and several additional libraries (scikit-learn version 1.8, pandas, scipy and matplotlib). This cross validation process ran solely on a laptop cpu and took under 5 min to run when processed on the complete dataset. All of the source code for this project can be found on Github page with link provided above.

V. Results

A. Main Results

Table I presents a summary of cross-validation for each model. Random Forest as the top metadata only model and linear svm as the top combined model had identical F1 scores at approximately 0.72 - 0.73 in regards to noise. Naive bayes is the top model based upon precision with a score of .95; however, it has a very low recall value. Similarly, Logistic Regression using text only was able to achieve the highest recall values 0.89, yet it also achieved only average precision.

TABLE I
Cross-validation results (stratified 5-fold). Highest model score by column shown in bold.

Model	Acc.	Prec.	Rec.	F1	AUC
Text-only					
Logistic Regression	0.949	0.485	**0.887**	0.626	0.978
Naive Bayes	0.970	**0.953**	0.402	0.565	0.948
Linear SVM	0.966	0.607	0.859	0.710	0.977
Metadata-only					
Decision Tree	0.921	0.363	0.815	0.502	0.889
Random Forest	0.974	0.748	0.704	**0.725**	0.973
Gradient Boosting	0.978	0.910	0.598	0.721	0.972
Combined					
Logistic Regression	0.942	0.450	**0.935**	0.608	**0.984**
Random Forest	0.964	0.594	0.782	0.675	0.981
Linear SVM	**0.967**	0.608	0.897	0.724	**0.986**

A comparison of F1 score is presented visually in Fig. 6. While linear models based on text or both text and metadata,

and tree-based group methods based on metadata have relatively comparable performance bands F1 $\approx$ 0.7), Decision Trees and Naive Bayes perform significantly worse than these two categories.

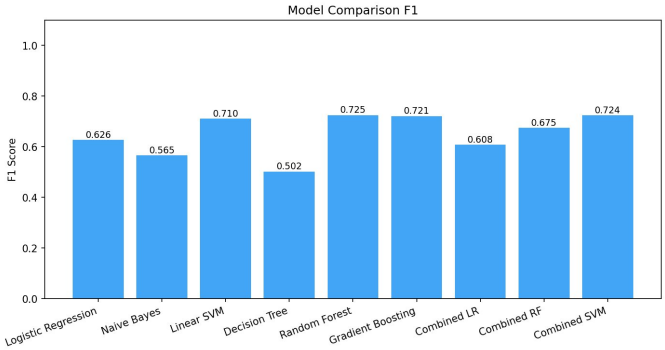

Fig. 6. F1 score comparison across all nine models.

The ROC Curves (Fig. 7) demonstrate that all three methods are well above a random threshold curve and the method using both modalities has an Area Under Curve (AUC) of 0.986 on the held out 20% test data.

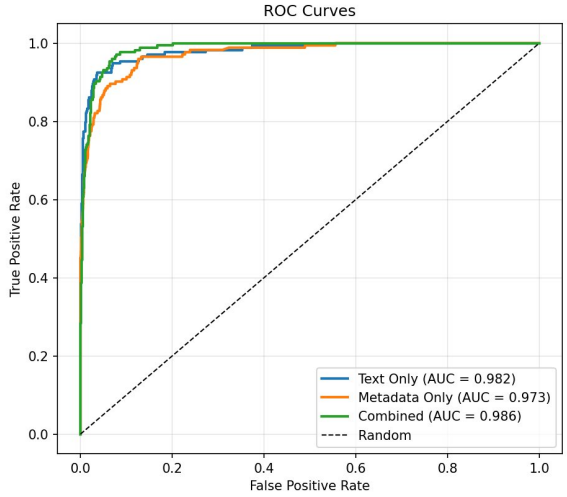

Fig. 7. ROC curves on the endured test set. The combined model achieves the highest AUC.

B. Ablation Study: Text vs. Metadata vs. Combined

We tested the contribution of each modality to classification by training the identical model (Logistic Regression with balanced class weights), but for the following three different sets of input variables: text only, metadata only and both together. The results are given in Fig. 8 as well as Table II.

Three key results found out that the performance of a linear model with only metadata as input is very poor (precision = 0.17). When incorporating metadata into the textual data improves both recall (from 0.89 to 0.94) and AUC (from 0.978 to 0.984), showing that the integrated approach is better able to detect fraudulent postings. The drop-off in precision caused

TABLE II
ABLATION (LOGISTIC REGRESSION ACROSS FEATURE SETS)

Features	Prec.	Rec.	F1	AUC
Text only	0.485	0.887	0.626	0.978
Metadata only	0.169	0.772	0.277	0.863
Combined	0.450	0.935	0.608	0.984

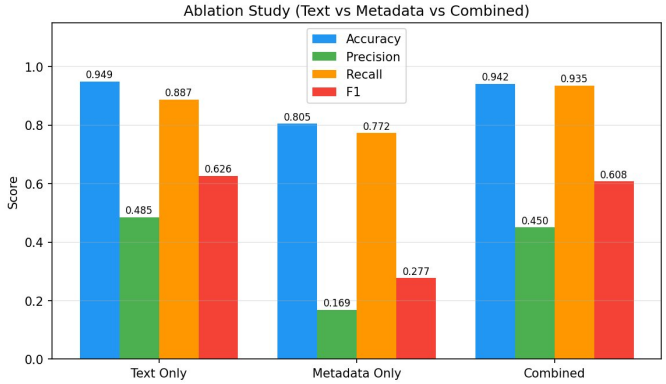

Fig. 8. Combining Features results in the best Recall and AUC for Logistic Regression (Ablation Study)

by combining metadata into the text-based learner reflects that the metadata characteristics identify far too many legitimate postings as fraudulent. In addition, since Tree-Based Learners handle such imbalances much better than Logistic Regression, it follows that the F1 value for the combined Linear SVM model shown in Table I is significantly higher.

C. Feature Importance

The Fig. 9 shows how important each of the features were to the Random Forest metadata model. Therefore, the first three most important features based on this data are `has_company_profile`, `location_enc` and `has_company_logo`. This finding supports our EDA.

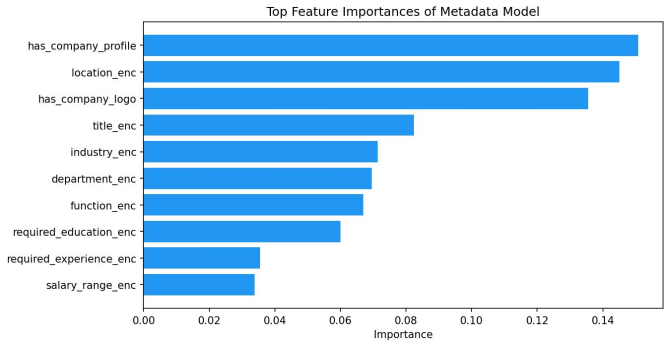

Fig. 9. Top metadata feature importances from the Random Forest model.

D. Confusion Matrices and Error Analysis

Confusion matrices for the 20% of the data used to evaluate performance is depicted in Fig. 10 and are provided below. We find that both text-only model and combined model each

capture 160 and 161 of the 173 fraudulent posts (recall = 0.92 and 0.93 respectively), while the metadata-only Random Forest captures 122 of the 173 (recall = 0.71) yet has the lowest number of false positives (48).

There is an obvious similarity among the fraud cases that were missed by all three models, in that they are well-formatted, lengthy postings that include typical corporate verbiage and have been populated for most of their metadata features. Each of these postings appear to be nearly identical to legitimate postings in our selected feature space; it is likely that capturing them would require some form of external signal such as sender reputation, posting rate or cross listing comparison which are not available within the current data.

VI. Discussion

The main results of this study are that ”text” and ”metadata” are useful for identifying job scams in ways that complement each other. This means that the ”text” part of the data identifies certain characteristics, e.g. vague promises ”work from home”, etc. While the ”metadata” part identifies structural or structural shortcuts, e.g., lack of company information, no logo, no salary range, which allow us to create a better model than we would have otherwise had. We can rely on either piece of information alone, since when we use either one individually, the model works well. But when we can combine them together, they work very well. In fact, when using both pieces of information collectively we find that the ROC-AUC value is nearly 0.99.

Our second key finding is that the highest possible F1 score we were able to obtain from the model was approximately 0.72 - 0.73. After achieving this level, the quality of the data begins to limit how good a model can become. There are essentially two types of errors. Type (i) involves fake postings that are created with great care so they appear to be legitimate postings. The model cannot identify these posting based on what model sees. Type (ii) includes postings that are legitimate/real postings, but they are somewhat unusual and therefore our model has identified them as suspicious postings. The primary issue here is related to the weights assigned to the classes, which can be partially addressed through precision-recall tuning of thresholds if there is a preference from a downstream platform for fewer false positives.

Limitations

- The data set comes from one source and has no labels for the fraud types contained within it; therefore, the model may not work well for newer types of scam that were developed after the date this data set was created.
- We did not use deep contextual natural language models. If we had used a pre-trained transformer fine-tuned on the description field in the data set, we believe it would have reduced some or all of the remaining gap between actual performance and optimal performance, particularly for fraud that mimics legitimate language.
- We treated ordinal-type categorical columns `required_education`, `required_experience` as if they were nominal type categorical columns.

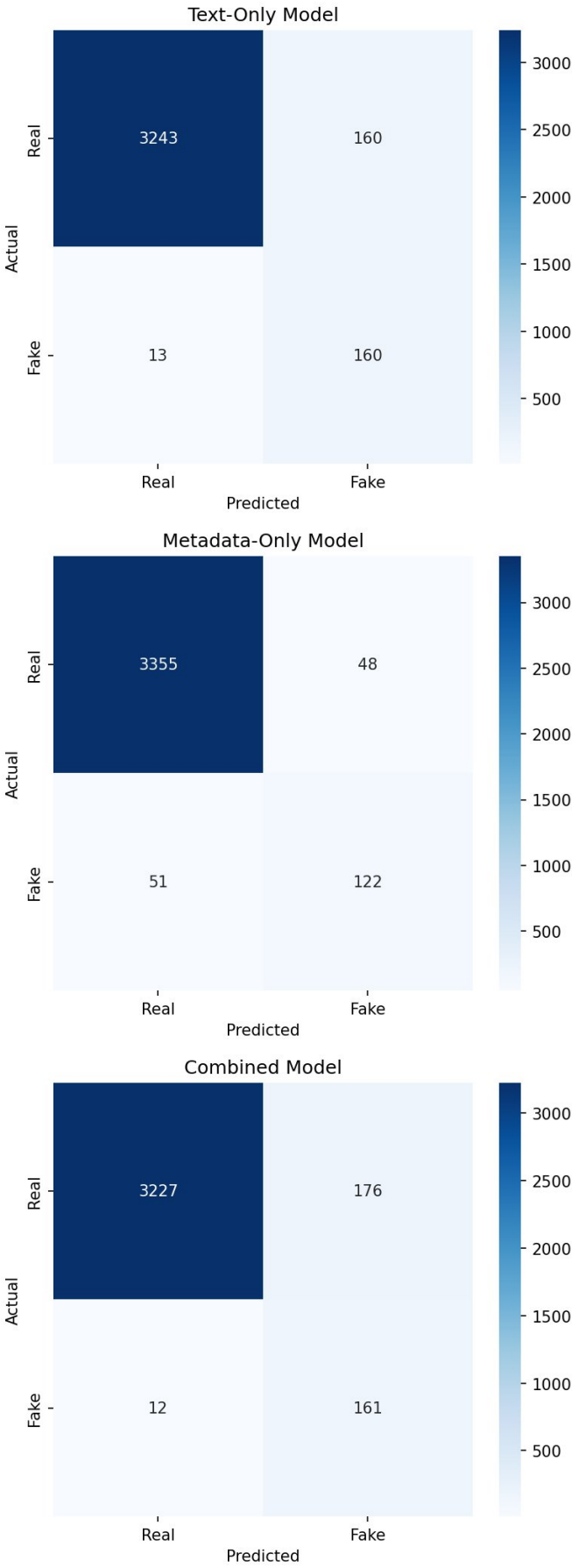

Fig. 10. Confusion matrices on the 20% held-out test set: text-only (top), metadata-only (middle), combined (bottom).

Treating these columns with an explicitly ordered encoding would have been a small but a well-justified improvement.

- Cross validation alone will provide overly optimistic estimates of how well a model performs at recalling fraudulent instances when there are only a few specific sub-populations where most of the fraud occurs (i.e. just one industry). To better estimate how well the model will perform across all possible sub-populations of interest, performing a leave one industry out cross validation test would have provided a more realistic view of the model's ability to generalize.

VII. Conclusion

We developed and assessed a fraud detection pipeline using the Real Or Fake job posting data set. We compared both text and metadata based classification models as well as the combination of those models with a single cross-validation procedure. In comparison, to both text-based and metadata-based models, our combined linear SVM classifier performed better than both. Our combined linear SVM model had an F1 score of 0.724 with an AUC-ROC score of 0.986. This was also slightly better than our metadata-based random forest classifier 0.725, and substantially better than our logistic regression text-based classifier 0.625. An additional study in which we isolated each modality to determine if they were redundant or complementary showed that the two modalities are complementary but not redundant. There is future potential for the inclusion of contextual language model features in the model, improved treatment of ordinal variables, and a model shift study where all of one industry's data would be held-out during model development.

Author Contributions

Yogeshwor Prasad Kharel handled the data loading, exploratory data analysis, text processing pipelines (text preprocessing and TF-IDF) for text models, the plots for model evaluations (ROC Curve and Confusion Matrix) and text-based model evaluations and handled the Error Analysis Module.

Akhalesh Sah handled the Metadata Preprocessing Pipeline (Encoding, Handling Missing Values, Engineering Features), Model Evaluations (Metadata Models), Label Imbalance Study.

The two authors worked together to develop the Combined Model, Cross Validation Utility, Column Auto-Detecting Module, Hyperparameters Tuning and Final Report.

References

[1] S. Bansal, "Real or Fake Job Posting Prediction," Kaggle Dataset, 2020. [Online]. Available: https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction

[2] S. U. Habiba, M. K. Islam, and F. Tasnim, "A Comparative Study on Fake Job Post Prediction Using Different Data Mining Techniques," in *Proc. 2nd Int. Conf. Robotics, Electrical and Signal Processing Techniques (ICREST)*, 2021, pp. 543–546.

[3] A. Amaar, W. Aljedaani, F. Rustam, S. Ullah, V. Rupapara, and S. Ludi, "Detection of Fake Job Postings by Utilizing Machine Learning and Natural Language Processing Approaches," *Neural Processing Letters*, vol. 54, no. 3, pp. 2219–2247, 2022.

[4] V. Vajjala et al., "Detecting Fake Job Postings Using Machine Learning Techniques," in *Proc. Int. Conf. Computational Intelligence and Data Engineering*, 2020.

[5] A. S. Pillai et al., "Detecting Fake Job Postings Using a Bidirectional LSTM and Ensemble Models," *Int. Research J. Modernization in Engineering, Technology and Science (IRJMETS)*, 2023. [Online]. Available: https://arxiv.org/pdf/2304.02019

[6] T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," in *Proc. European Conf. Machine Learning (ECML)*, 1998, pp. 137–142.

[7] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[9] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.